

Relatório de Execução — Roteiro 2

Central de Avisos da Turma · Revisão de Redes de Computadores (U0)

Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas · 16 de agosto de 2026

1. Resumo

As quatro partes do roteiro foram implementadas nas duas linguagens, totalizando os **8 programas** exigidos, e **todos foram compilados e executados com sucesso** nesta máquina. As 12 questões do roteiro estão respondidas em `RESPOSTAS.md`, com as respostas fundamentadas no comportamento observado — três delas foram verificadas com testes dedicados, e não apenas descritas pela teoria.

Parte	Java	Python	Verificação realizada
A — TCP	OK	OK	3 mensagens trocadas + comando <code>hora</code> respondendo o horário do servidor
B — UDP	OK	OK	troca normal + envio com o servidor desligado
C — Multicast	OK	OK	2 clientes simultâneos recebendo os mesmos 5 avisos
D — WebSocket	OK	OK	2 clientes; mensagem de um aparecendo no outro em tempo real

Pendências para o aluno concluir a entrega:

1. **Os 8 prints de tela** da pasta `evidencias/`. São a evidência da execução do próprio aluno e precisam ser capturados por ele, com o `Get-Date` visível conforme a seção 3.2. O script `scripts/demo.ps1` abre os terminais já prontos para isso.
2. **Definir o OFFSET** das portas (seção 3.3) com os dois últimos dígitos do RA. Está em `0` em todos os arquivos.
3. **Configurar a identidade do Git** (`user.name` / `user.email`), ainda não definida nesta máquina — ver seção 7.

2. Ambiente

Ao iniciar, verificou-se que **nem o Java nem o Python estavam instalados** nesta máquina, apesar de existirem arquivos `.class` versionados no repositório. Ambos foram instalados via `winget`.

Componente	Versão	Observação
Sistema	Windows 10 Pro 19045	PowerShell como terminal
OpenJDK	21.0.12 (Microsoft)	instalado durante esta execução
Python	3.12.10	instalado durante esta execução
websockets	17.0.1	<code>pip install websockets</code>
Java-WebSocket	1.5.6	+ <code>slf4j-api</code> e <code>slf4j-simple 2.0.9</code>
Maven	indisponível	não existe pacote no winget; ver abaixo

Como o Maven não pôde ser instalado, a Parte D em Java seguiu a **alternativa oficial da seção 7.2 do roteiro** (download manual dos `.jar` para `java/websocket/lib/` e compilação via `-cp "lib/*"`). O `pom.xml` exigido pelo roteiro está no repositório e correto — apenas não foi o caminho usado para compilar.

3. Ponto de partida

O repositório continha apenas dois commits ("first commit"), um `README.md` de uma linha, um `RESPOSTAS.md` vazio e a Parte A em Java na versão original do roteiro — **sem** a modificação do comando `hora` pedida na tarefa 4.5.3, e com os `.class` versionados indevidamente. Não havia nenhum arquivo Python, nenhuma das partes B, C e D, nem `.gitignore` ou pasta de evidências.

4. Resultados dos testes

Todas as saídas abaixo são reais e estão gravadas em `logs/`.

Parte A — TCP

O servidor responde ao comando `hora` com o horário atual, conforme exigido:

```
[servidor]                                     [cliente]
[TCP] Cliente conectado: /127.0.0.1           [TCP] Conectado ao servidor.
[TCP] Recebido: Ola monitor, tudo bem?        > Monitor responde: recebi sua mensagem -> "Ola monitor, tudo bem?"
[TCP] Recebido: Qual a sala da prova?         > Monitor responde: recebi sua mensagem -> "Qual a sala da prova?"
[TCP] Recebido: hora                          > Monitor responde: agora sao 16/08/2026 21:59:31
[TCP] Recebido: sair                          > Encerrando conexao. Ate mais!
```

Parte B — UDP

Com o servidor desligado, o comportamento **diferiu entre as linguagens**, o que rendeu material para a questão B1: em ambas o *envio* teve sucesso, mas o Java **travou indefinidamente** na espera da resposta enquanto o Python **lançou exceção**.

```
Java - cliente sem servidor
[UDP] Pronto para enviar. Digite 'sair' para encerrar.
> (processo encerrado por timeout apos 15 s, sem erro e sem resposta)

Python - cliente sem servidor
ConnectionResetError: [WinError 10054] Foi forçado o cancelamento de uma conexao existente
```

A causa está explicada em `RESPOSTAS.md`: o Windows devolve um ICMP *port unreachable*, que o Python repassa à aplicação e o Java ignora em sockets não-conectados.

Parte C — Multicast

Funcionou nas duas linguagens, com os dois clientes recebendo os cinco avisos. O ponto a observar é a assimetria: o servidor imprimiu **5 envios** e houve **10 recebimentos** — uma única transmissão alcançando o grupo inteiro.

```
[servidor]                                     [cliente 1 e cliente 2, simultaneamente]
[Multicast] Enviado: Aviso #1 ... 4 minuto(s)! [Multicast] Recebido: Aviso #1 ... 4 minuto(s)!
[Multicast] Enviado: Aviso #2 ... 3 minuto(s)! [Multicast] Recebido: Aviso #2 ... 3 minuto(s)!
[Multicast] Enviado: Aviso #3 ... 2 minuto(s)! [Multicast] Recebido: Aviso #3 ... 2 minuto(s)!
[Multicast] Enviado: Aviso #4 ... 1 minuto(s)! [Multicast] Recebido: Aviso #4 ... 1 minuto(s)!
[Multicast] Enviado: Aviso #5 ... 0 minuto(s)! [Multicast] Recebido: Aviso #5 ... 0 minuto(s)!
```

Parte D — WebSocket

O mural funcionou nas duas linguagens: a mensagem publicada por um cliente chegou ao outro sem que este tivesse solicitado nada, comprovando o *push* do servidor.

```
[servidor Python]                               [cliente 2 – nao enviou nada ainda]
[WebSocket] Novo aluno conectado. Total: 1      Bem-vindo(a) ao mural de avisos da turma!
[WebSocket] Novo aluno conectado. Total: 2      Aviso da turma: Pessoal, a prova foi adiada para sexta
[WebSocket] Recebido: Pessoal, a prova ...      Aviso da turma: Valeu pelo aviso, monitor!
[WebSocket] Aluno desconectado. Total: 0
```

5. Problemas encontrados e correções

Três defeitos reais apareceram durante a execução e foram corrigidos no código:

5.1 O servidor UDP em Python caía quando um cliente saía

Quando o cliente encerrava, a resposta do servidor batia numa porta já fechada e o ICMP de retorno **derrubava o servidor** com `WinError 10054` na iteração seguinte — um servidor sem conexão não pode morrer porque um cliente sumiu. O `recvfrom` passou a tratar `ConnectionResetError` e continuar o laço.

5.2 O cliente multicast em Java não se inscrevia de forma confiável

O roteiro usa `NetworkInterface.getByInetAddress(InetAddress.getLocalHost())`, que escolhe uma única interface e falha quando a máquina tem adaptadores virtuais — esta tem **12 deles** (Hyper-V/WSL). O cliente passou a se inscrever no grupo em *todas* as interfaces ativas com suporte a multicast, e assim funcionou (2 interfaces aceitaram a inscrição).

5.3 Arquivos .class versionados

Os binários compilados da Parte A estavam no controle de versão. Foram removidos com `git rm --cached` e um `.gitignore` foi criado conforme a seção 3.4.

6. Verificação empírica das questões

Três questões descrevem comportamentos que poderiam ser respondidos apenas pela teoria. Para sustentar as respostas, cada uma foi reproduzida em teste próprio:

Questão	Teste	Resultado observado
A1	cliente iniciado sem servidor	<code>ConnectException: Connection refused</code> (Java) e <code>ConnectionRefusedError [WinError 10061]</code> (Python), ambos já na abertura do socket
A3	dois clientes no mesmo servidor	o cliente B conectou (imprimiu "Conectado ao servidor", pois o handshake TCP é feito pelo SO e a conexão ficou na fila de <i>backlog</i>), mas nunca foi atendido : enviou sua mensagem e ficou travado sem resposta. O servidor registrou um único "Cliente conectado" e encerrou junto com o cliente A
C3	cliente entrando no grupo com atraso	recebeu apenas os avisos #4 e #5 ; os avisos #1 a #3, enviados antes da inscrição, foram perdidos definitivamente

O caso A3 é o mais instrutivo: a resposta ingênua seria "o segundo cliente não consegue conectar", e o teste mostra que ele *conecta* — a limitação está na aplicação, que chama `accept()` uma única vez, e não no protocolo.

7. Situação do repositório

O trabalho está organizado em **7 commits incrementais** — um por parte, mais a configuração inicial, os scripts e a documentação — atendendo ao critério de granularidade do roteiro. **Os commits estão preparados mas ainda não gravados:** o Git desta máquina não tem `user.name` e `user.email` configurados e recusa a operação. Como a autoria fica registrada permanentemente no histórico e é um dos critérios de avaliação, os dados precisam ser informados pelo aluno:

```
git config --global user.name "Seu Nome Completo"
git config --global user.email "seu.email@aluno.pucminas.br"
```

8. Checklist do roteiro

Situação	Item	Observação
Feito	Estrutura de pastas	conforme a seção 3.1, mais <code>scripts/</code> e <code>logs/</code>
Feito	<code>.gitignore</code>	seção 3.4, com <code>out/</code> e os <code>.jar</code> acrescentados
Feito	8 soluções compilando e executando	4 protocolos × 2 linguagens, todas verificadas
Feito	Comando <code>hora</code> (tarefa 4.5.3)	nos servidores TCP das duas linguagens
Feito	<code>RESPOSTAS.md</code> com as 12 questões	respostas fundamentadas no comportamento observado em execução
Feito	Histórico incremental	7 commits, padrão <code>tipo(escopo): descrição</code>
Pendente	8 prints em <code>evidencias/</code>	capturar com <code>scripts/demo.ps1</code> , com <code>Get-Date</code> visível
Pendente	<code>OFFSET</code> das portas	trocar <code>0</code> pelos 2 últimos dígitos do RA
Pendente	Identidade do Git	necessária para gravar os commits
Pendente	Publicação do repositório	conforme orientação do professor

Antes de entregar. O roteiro cobra que o aluno saiba explicar e defender qualquer trecho entregue. Vale revisar com atenção as questões A3, B1 e C3, que discutem comportamentos observados em execução, e as três correções da seção 5 deste relatório, que fogem do código original do roteiro e são candidatas naturais a pergunta em sala.